MyCodeSchool

Code review — stefan-colections-csharp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 5 — 2026-10-28 — sha `b6d2949`

Scop: fix-urile din runda 4 (cele 6 🔴), din commit-ul `b6d2949` „code review fixes".

Metodă: **build-ul nu trece** (B1). Ca să văd restul, am făcut pe o **copie** minimul ca să compileze (am pus la loc constructorul `PairNode(K key, V value)`), apoi am rulat `Main` și ~60 de teste mici: `Remove` pe tablou plin în toate clasele, `Insert`, `Count` la mulțime, `TryToTake`, `Add` și `this[k] = v`, `foreach` cu modificări pe `Keys` și pe `Values`. **Toate constatările de mai jos în afară de B1 sunt verificate pe copia cu B1 reparat minimal**; codul tău nu a fost atins.

Poarta de compilare: ❌ 1 eroare (`CS1729`), 24 de warning-uri (runda 4: 35).

Pe scurt:

- ✅ **Din cele 6 🔴 din runda 4, trei sunt închise pe bune:** `Remove` pe tablou plin (în toate cele 4 clase), `TryToTake` nu mai șterge, `Add` pe cheie existentă aruncă. 👏 Plus `Contains` fără copie (M2), `Pozitie` cu hash bun, mesajele dicționarului cu punct.
- ⚠️ Trei sunt **pe jumătate**: `Insert` pe lista goală, `Count` la mulțime, fail-fast.
- 🔴 Și au apărut **trei lucruri noi, stricate chiar de fix-uri**: `Insert(0)` pe o listă cu elemente o golește (B2), `d[k] = v` pe cheie nouă aruncă (B3), iar `foreach` pe `Values` tot nu prinde nimic (B5). Din cauza lor, `Main` se oprește în Pas 1 (B2); cu B2 reparat și el, se oprește la Pas 5c (B3), deci Pas 6 și 7 nu rulează.
- Tiparul rundei: repari rândul semnalat, iar fix-ul îl strică pe vecin sau lasă gemenul neatins (`Keys` reparat, `Values` nu). **Rulează `Main` după fiecare schimbare, nu doar la sfârșit.**

Cele 6 🔴 din runda 4, prin rulare

r4	Ce am rulat	Stare
B1 `Remove` pe tablou plin	4, 8, 16 elemente, în `ListaTablou`, `StivaTablou`, `CoadaCirculara`, `ListaInlantuita`; și coadă rotită	✅ închis, în toate 4
B2 `Insert` pe lista goală	`Insert(5, "x")` și `Insert(-3, "x")` pe goală → `ArgumentOutOfRangeException`	⚠️ garda merge, dar a stricat `Insert(0)` (B2 nou)
B3 `Count` la mulțime	`Remove(99)` → `False`, `Count` rămâne 2; `Clear` → 0	⚠️ `Count` e bun doar pe tipul `MultimeHash`; `ToString` crapă (B4 nou)
B4 `TryToTake`	`TryToTake("Maria")` → `True 18`, cheia rămâne, `Count=1`	✅ închis
B5 `Add`	`Add("Ana", 18)` → `ArgumentException`, `d["Ana"]` rămâne 17	✅ `Add`; ⚠️ dar `d[k] = v` a stricat (B3 nou)
B6 fail-fast	`Keys` + `Remove(0)` + `Add(6)` → `InvalidOperationException`; `Keys` + `d["a"] = 100` → idem	✅ `Keys`; 🔴 `Values` neatins (B5 nou)

🔴 Critice

B1 — `DictionarHash.cs:64`, `Common/PairNode.cs`: nu compilează (`CS1729`)

Eroarea: `DictionarHash.cs(64,43): error CS1729: 'PairNode<K, V>' does not contain a constructor that takes 2 arguments`. Restul erorilor nu există; e una singură.

DE CE: `CS1729` înseamnă „ai cerut un constructor cu 2 argumente, iar clasa nu are unul". Rândul `new PairNode<K, V>(key, value)` cere constructorul care primește cheia și valoarea. În runda 4 ți-am

spus că `PairNode()` (cel gol) și câmpul `next` sunt cod mort; în `b6d2949` ai șters `next`, dar din cele **două** constructoare l-ai șters pe cel folosit, `PairNode(K key, V value)`, și l-ai păstrat pe cel gol, pe care nu-l cheamă nimeni. Cât timp clasa avea măcar un constructor scris de tine, C# nu mai pune singur unul gol, deci nu era altă cale să se construiască perechea.

💡 Care din cei doi constructori îl cheamă `Add` (`:64`)? Dacă ștergi unul, caută întâi cine îl folosește. Și un obicei: `dotnet build` înainte de fiecare commit.

B2 — `ListaInlantuita.cs:88` : `Insert(0, x)` pe o listă cu elemente o golește

Dovada: `[b, d]` → `Insert(0, "a")` → `[a, , ] Count=3` : `b` și `d` au dispărut, iar `Count` crede că sunt 3. Din cauza asta `Main` pică în `DemoLista` (`Program.cs:34`, `Insert(4, "e")`) și nu ajunge la Pas 2. Pe lista goală garda merge acum (`Insert(5)` și `Insert(-3)` aruncă).

DE CE: ca să muți garda pe primul rând, ai șters ramura `Head == null` și ai copiat în ramura `index == 0` rândul ei: `new Node<T>(item, null)`. Acel `null` e „nodul de după mine": pe lista goală e corect, pe lista cu elemente taie lanțul de la început, iar `Head` arată spre un nod singur. Înainte aveai `new Node<T>(item, Head)`, care mergea **și** pe lista goală, fiindcă acolo `Head` e chiar `null`. Ai uitat doar un lucru pe lista goală: `Tail` trebuie și el pus pe nodul nou.

💡 Ce rând îl pui după `Head = new Node<T>(item, Head)` ca `Tail` să fie corect doar când lista era goală?

B3 — `DictionarHash.cs:31` : `d[k] = v` pe o cheie nouă aruncă

Dovada: `d["Nou"] = 5` → `KeyNotFoundException`. Din cauza asta `Main` se oprește la Pas 5c (`Program.cs:190`, `tabla[rege] = "rege"`), deci nici Pas 6, nici Pas 7 nu mai rulează.

DE CE: ai separat corect cele două uși (B5 din runda 4), dar ai făcut ușa `d[k] = v` să se poarte ca `d[k]` la citire: „cheia nu există → eroare". Contractul scrierii e altul: **înlocuiește sau adaugă** (fișa, Pas 5: „scrierea înlocuiește sau adaugă"). Excepția la cheie lipsă e doar la **citire**; la `Add` e la cheie **existentă**; la scriere nu există nicio excepție. Ai acum o gardă care trebuie întoarsă: dacă perechea există, schimbi `Value`; dacă nu, o adaugi ca la `Add`.

💡 `Add` face deja „pune în găleată, `Count++`, `versiune++`, redimensionează". Cum poate folosi indexerul același cod, fără excepția de la `Add`? (O metodă `private`.) Verifică și că `Find` nu se cheamă de două ori.

B4 — `MultimeHash.cs:9`, `Colectii/ColectieAbstracta.cs:9` : `Count` ascuns, `ToString` crapă

Dovada: `{1, 2}` → `ToString()` și `ToArray()` → `IndexOutOfRangeException` la `ColectieAbstracta.cs:34`. `m.Count` pe `MultimeHash` dă 2, dar prin `ColectieAbstracta<int>` dă

0. `Reuniune(...).ToString()` crapă la fel. `lista.Count = 10` merge în continuare (`ListaTablou` → crapă la afișare, `ListaInlantuita` → `[1, 0, 0, ...]`).

DE CE: `Count` din mulțime a fost mutat pe `dictionar.Count` (bine, un singur număr!), dar scris `public int Count => ...` **fără** `override`. Compilatorul te avertizează (`CS0108`, în cele 24 de warning-uri) că **ascunzi** `Count` din bază, nu îl înlocuiești. Cine are tipul `MultimeHash` vede noul `Count`; codul din bază (`ToArray`, rândul `:31`) vede în continuare câmpul `count` din bază, care e mereu 0. Rezultat: `ToArray` face un tablou de 0 locuri, iar `foreach` îi pune 2 elemente. E același mecanism ca la `Contains` în runda 4 (M2): ascunderea alege după tipul **variabilei**, suprascrierea după tipul **obiectului**. Iar `set` public din bază lasă pe oricine să mintă `Count`-ul.

💡 Fișa, Pas 3: `Count` e `abstract` în bază, deci fiecare clasă îl dă în felul ei. Ce ar rămâne în bază din `count` și `set`? Cine are nevoie de câmp (`ListaTablou`?) și cine nu (`MultimeHash`)?

B5 — `DictionarHash.cs:139`: `foreach` pe `Values` nu prinde nicio modificare

Dovada: `{"a": 1, "b": 2}`, `foreach` pe `Values` cu `d["a"] = 100` în corp → **fără excepție**. Pe `Keys` aceeași modificare aruncă corect. Pe `Values` cu `Remove` aruncă doar din întâmplare (vezi mai jos).

DE CE: rândul `if (versiune != versiune)` compară un număr cu el însuși, deci e **mereu fals**. În `Keys` ai redenumit copia locală în `index` și ai scris `versiune != index`; în `Values` ai schimbat primul rând, dar nu și al doilea (`int index = versiune;` e la `:132`, nefolosit). Acolo unde totuși vezi excepție la `Values` + `Remove`, aruncă `ListaInlantuita` din găleată, cu contorul ei, nu dicționarul (cum am explicat la B6 din runda 4). Aceeași greșeală de tip „repar un rând, gemenul lui rămâne" ca la `Remove`/`RemoveAt` în runda 3.

💡 Ce două rânduri din `Keys` (`:114`, `:121`) trebuie să se regăsească identic în `Values`?

🟡 Importante

- **M1** — ⏸ numele din contract (`Add`/`Remove`/`TryToTake` vs `Adauga`/`Sterge`/`IncearcaSaIa`): de decis la ședință.
- **M2** — ✅ închis: `Contains` din bază merge pe `foreach`, mulțimea îl suprascrie (`override`); 100.000 de elemente alocă 40 de octeți (era 390 KB), 200 de `Contains` prin bază: 0,008 ms (era 2164 ms).
- **M3** — mesajul „Collection was modified; enumeration operation may not execute" n-are punct la final în 4 locuri: `ListaTablou.cs:124`, `ListaInlantuita.cs:233`, `StivaTablou.cs:89`,

`CoadaCirculara.cs:78`. În `DictionarHash` ✅, iar `KeyNotFoundException` are acum textul din .NET ✅. Repară cele 4, nu doar una.

- **M4** — `Pozitie.cs:17`: `Equals(null)` și `Equals("x")` → `NullReferenceException` (`as` dă `null` pe alt tip). `Equals` nu aruncă niciodată, întoarce `false`. ✅ `GetHashCode` cu `HashCode.Combine`: 64 de hash-uri diferite pe 64 de poziții.
- **M5** — warning-uri 35 → 24 ✅ (`buckets` nu mai e `?`). Rămas: `Node<T>` / `PairNode<K,V>` `public`, cu câmpuri publice; 24 de warning-uri cer un `dotnet build` citit, nu ignorat (un `CS0108` te-ar fi avertizat la B4).

🟢 Cleanups

- **C1** — `PairNode.cs:7`: constructorul gol e tot cod mort (vezi B1); `next` ✅ și câmpul `node` ✅ scoase.
- **C2** — ✅ închis: `Reuniune` fără `Contains` în plus.
- **C3** — `ExYield` (`Program.cs:7`) și `//IColectie`, `//ILista`, `//ICoada` (`ListaTablou.cs:11`, `:48`; `CoadaCirculara.cs:12`, `:44`; `StivaTablou.cs:11`) în cod; `RASPUNSURI.txt` lipsește în continuare.

Before / After

	Acum	Corect
B1	doar `public PairNode()`	`public PairNode(K key, V value) { Key = key; Value = value; }`
B2	`Head = Tail = new Node<T>(item, null);`	`Head = new Node<T>(item, Head);` + `Tail` doar dacă lista era goală
B3	`if (!ContainsKey(key)) throw ...` în `set`	`set`: dacă perechea există → `Value = value`, altfel adaugă ca `Add`
B4	`public int Count => dictionar.Count;`	`public override int Count => dictionar.Count;` cu `Count` `abstract` în bază
B5	`if (versiune != versiune)`	`if (versiune != index)`

Q&A

1. `CS1729` și `CS0108` sunt amândouă mesaje ale compilatorului pe care le-ai primit. Care e eroare, care e avertisment, și de ce al doilea te-ar fi salvat de B4?
2. La `Insert(0, x)`, de ce mergea `new Node<T>(item, Head)` și pe lista goală, deși acolo `Head` e `null`?
3. `d["x"]` (citire) aruncă pe cheie lipsă, `d.Add("x", 1)` aruncă pe cheie existentă. Când aruncă `d["x"] = 1`?

Răspunsurile (și cele 9 din rundele 1–3) în `RASPUNSURI.txt`, în repo.

Următorul pas: B1 (build verde), apoi `Main` rulat întreg: Pas 1 până la Pas 7 trebuie să iasă identic cu fișa; apoi M3, M4 și `RASPUNSURI.txt`.

Runda 4 — 2026-10-27 — sha `65e9167`

Build pe .NET 8 ✅ (0 erori, 35 warning-uri); Pas 1–4 identice cu fișa.

- ✅ **B1** `ListaTablou.Remove` golea `items[Count]`: închis, în toate 4 clasele (→ runda 5)
- ⚠️ **B2** `Insert` pe lista goală ignora indicele: garda merge · 🔴 `Insert(0)` pe listă nevidă o golește (→ runda 5, B2)
- ⚠️ **B3** `Count` ținut de mână în `MultimeHash`: `Remove` / `Clear` ✅ · 🔴 `Count` ascuns, nu suprascris; `set` public rămas (→ runda 5, B4)
- ✅ **B4** `TryToTake` șterge cheia: închis
- ⚠️ **B5** `Add` pe cheie existentă suprascria: `Add` ✅ · 🔴 `d[k] = v` pe cheie nouă aruncă (→ runda 5, B3)
- ⚠️ **B6** fail-fast pe `Count`: `Keys` cu `versiune` proprie ✅ · 🔴 `Values` compară `versiune != versiune` (→ runda 5, B5)
- ⏸ **M1** numele din contract: de decis la ședință
- ✅ **M2** `Contains` fără copie + `override` în mulțime: închis
- ⚠️ **M3** mesaje: dicționarul ✅ · 4 locuri fără punct (→ runda 5, M3)
- ⚠️ **M4** `Pozitie`: hash ✅ · `Equals(null)` crapă (→ runda 5, M4)
- ⚠️ **M5** warning-uri 35 → 24 ✅ · nod public rămas (→ runda 5, M5)
- ✅ **C1** cod mort în dicționar (`node`, `next`): închis · ⚠️ constructorul gol rămas (→ runda 5, C1)
- ✅ **C2** `Reuniune` cu `Contains` în plus: închis · 🔴 **C3** notițe în cod, `RASPUNSURI.txt` lipsă

Runda 3 — 2026-10-20 — sha `2db692a`

Fix-urile din runda 2 + Pas 3 (`ColectieAbstracta`), Pas 4 (stiva, coada), început de Pas 5.

- ✅ **B1** nu compila: închis, `Main` cheamă toți pașii
- ✅ **B2** două `Count` -uri: închis · 🔴 `set` public rămas (→ runda 4, B3)
- ✅ **B3** `ToArray` întorcea tabloul din interior: închis (scris o dată, în bază)
- ✅ **B4** golirea locului: închis în `RemoveAt` , garda `>=` reparată · 🔴 repetat în `Remove` (→ runda 4, B1)
- ✅ **B5** tabloul creștea cu 1: închis în toate trei clasele, `Capacity => items.Length`
- ✅ **B6** stiva parcursă de la bază: închis
- ⏸ **M1** numele din contract: de decis la ședință (→ runda 4, M1)
- ⚠️ **M2** Template Method: `ToArray` din `foreach` ✅, `GetEnumerator` ne-generic scos ✅ · `Contains` tot din `ToArray` (→ runda 4, M2)
- 🔴 **M3** nod public + warning-uri: 25 → 35 (→ runda 4, M5)
- ⚠️ **M4** index greșit: `ListaTablou` închis · 🔴 `ListaInlantuita.Insert` pe lista goală (→ runda 4, B2)
- ⚠️ **M5** nodul dicționarului: `PairNode` cu cheie + valoare ✅ · câmpul `node` rămas (→ runda 4, C1)
- ✅ **C1** `Tail = new Node<T>()` : închis · ✅ **C2** `", "` : închis · ⚠️ **C3** `CoadaCirculara` cu `private` ✅, notițe în cod + `RASPUNSURI.txt` lipsă (→ runda 4, C3)

Runda 2 — 2026-10-17 — sha `a8e794b`

Fix-urile din runda 1 + Pas 2, `ListaTablou<T>` .

- ✅ **B1** `ToString` dubla primul element: închis (mutat în `ColectieAbstracta`)
- ✅ **B2** `AsVector` cu `Count + 1` : închis în `ListaInlantuita` · 🔴 repetat în `ListaTablou.ToArray` (→ runda 3, B3)
- ✅ **B3** `Tail = Head` în `Remove` : închis
- ✅ **B4** `GetEnumerator` ne-generic în `ListaTablou` : închis
- 🔴 **B5** `Capacity` câmp separat de tablou: neînchis, mai grav (→ runda 3, B5)
- ✅ **B6** `Equals` pe `null` : închis, în toate cele 4 clase
- ⏸ **M1** numele din contract: de decis la ședință (→ runda 3, M1)

- 🔴 **M2** nod public + warning-uri: 14 → 25 (→ runda 3, M3)
- 🔴 **M3** index greșit tăcut: neatins (→ runda 3, M4)
- ✅ **M4** dublare scrisă de 2 ori, bucle de căutare: închis (`ResizeIfFull`)
- ⚠️ **C1** `Node()` + `Tail` gol în constructor: rămas · ⚠️ **C2** golirea locului: scrisă, dar pe indicele greșit (→ runda 3, B4) · 🔴 **C3** `RASPUNSURI.txt` lipsă

Runda 1 — 2026-10-16 — sha `ac84a76`

Pasul 1: `IColectie<T>`, `ILista<T>`, `ListaInlantuita<T>`.

- ✅ **B1** `foreach` / `GetEnumerator`: închis · ⚠️ `ToString` scris, dar dublează primul element (→ runda 2, B1)
- ✅ **B2** `RemoveAt(0)` scădea `Count` de două ori: închis · ⚠️ ascunderea lui prin `Count + 1` în `AsVector` a rămas (→ runda 2, B2)
- ✅ **B3** `Tail` după constructor, `Clear`, `Insert(0)`: închis · 🔴 o linie nouă greșită în `Remove` (→ runda 2, B3)
- ✅ **B4** listă goală și listă cu un element: închis
- ✅ **M1** `Head` folosit drept cursor: închis
- ⏸ **M2** numele din contract: de decis la ședință (→ runda 2, M1)
- 🔴 **M3** nod public + warning-uri: neatins, 13 → 14 (→ runda 2, M2)
- 🔴 **M4** index greșit tăcut: neatins (→ runda 2, M3)
- ✅ **C1** `using`-uri nefolosite · ⚠️ **C2** `Node()` rămas · 🔴 **C3** notițe în cod, `RASPUNSURI.txt` lipsă